Text-to-Constraints: Mapping Regulatory Requirements to Executable Data Quality Constraints

Eduardo H. M. Pena
Federal University of Technology-Paraná
Brazil, Campo Mourão
eduardo@utfpr.br

Junio de Freitas
Federal University of Amazonas
Brazil, Manaus
junio.freitas@icomp.ufam.edu.br

Eduardo Cunha de Almeida
Federal University of Paraná
Brazil, Curitiba
eduardo@ufpr.br

Altigran da Silva
Federal University of Amazonas
Brazil, Manaus
alti@icomp.ufam.edu.br

ABSTRACT

Many regulatory documents specify requirements that valid data must satisfy, but those specifications are rarely available as data quality constraints. Data profiling, particularly constraint discovery, is a well-studied alternative when constraints are unavailable, but discovered constraints reflect what the data (approximately) satisfies rather than what it is required to satisfy. We introduce the *Text-to-Constraints* problem: given a dataset and related regulatory text, map the requirements stated in that text to executable data quality constraints over the dataset schema. Then, we present the first framework to address this problem, `Norma`, which identifies verifiable requirements in regulatory documents and maps them to denial constraints using a rich schema-grounded predicate vocabulary. We evaluate `Norma` on six datasets with diverse textual and data characteristics, covering constraints ranging from simple checks to complex cross-record rules. `Norma` outperforms baselines from data profiling and Text-to-SQL in constraint classification and extraction on all six datasets, recovering constraints they miss and identifying real regulatory violations in publicly released data, including two large mortgage datasets with up to one million records.

PVLDB Reference Format:
Eduardo H. M. Pena, Junio de Freitas, Eduardo Cunha de Almeida, and Altigran da Silva. Text-to-Constraints: Mapping Regulatory Requirements to Executable Data Quality Constraints. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/EduardoPena/norma-text-to-constraints.

1 INTRODUCTION

Many data management tasks rely on *data quality constraints*, which define what constitutes valid data. However, such constraints are often missing, incomplete, or outdated in real-world datasets. This gap has motivated a large body of work on *data profiling*, where data constraints are automatically discovered from the data itself [2]. Profiling algorithms extract constraints, such as functional dependencies (FDs) and denial constraints (DCs), by identifying patterns that hold over a given data instance, and have been widely used to support data cleaning and error detection [56].

However, this line of work derives constraints only from the data. Exact discovery returns rules that the instance satisfies, approximate discovery tolerates a fraction of violations [38, 44], and rule-discovery systems help experts rank and refine the candidates [19]. In all cases, the candidates are patterns that hold, exactly or approximately, in the data, rather than rules that the data is *required to satisfy* by some policy, regulation, or legislation. A required rule that is widely violated in the released data is therefore hard to recover from the data alone.

Many datasets are governed by rules specified in regulatory documents, policy manuals, and operational specifications, which define requirements such as mandatory fields, valid values, eligibility criteria, uniqueness conditions, quotas, and ordering rules [7, 12, 54]. Formalizing these rules is difficult because they depend on combinations of attributes, relationships across tuples, thresholds, and context-dependent conditions. Since they are typically expressed only in natural language, they are rarely translated into executable constraints and used with the data they govern. Consequently, datasets are frequently released without an explicit representation of their own correctness requirements, and violations remain undetected.

Example 1 (Brazilian public university admission). *Brazil's "Sistema de Seleção Unificada" (SISU) manages the public university placements for millions of students annually. The application data is publicly available,*[1] *with sensitive fields anonymized. The dataset schema includes attributes such as* student_id, course_code, shift, institution_id, year, *and* edition. *The rules that define correct enrollment outcomes, however, are described in separate documents that do not accompany the dataset. For instance, Normative Ruling No. 21/2012*[2] *is a PDF file found on the Brazilian government website that states (Article 19):*[3]

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

[1] https://dadosabertos.mec.gov.br/sisu
[2] https://sisugestao.mec.gov.br/docs/portaria-2012-21.pdf
[3] Schema elements and quoted statements translated from Portuguese for readability.

> *"... the student will be ranked in descending order of scores for the seat option they applied to, respecting the available seats at the institution, by campus location, course, and shift, as well as the student's admission category."*

This passage defines an ordering constraint*: within each context defined by institution, campus, course, shift, and admission category (and per year/edition), a higher score should not result in a worse rank. The following SQL query checks for violations of this requirement in SISU's application data:*

```
SELECT t.student_id, t'.student_id
  FROM applications t, applications t'
 WHERE t.student_id <> t'.student_id
   -- same context
   AND t.year = t'.year
   AND t.edition = t'.edition
   AND t.institution_id = t'.institution_id
   AND t.campus_id = t'.campus_id
   AND t.course_code = t'.course_code
   AND t.shift = t'.shift
   AND t.application_mode = t'.application_mode
   -- violation: higher score but worse ranking
   AND t.score > t'.score
   AND t.ranking > t'.ranking;
```

Any nonempty result set for this query indicates a violation of a legal requirement. On the released data, the query returns 76 violating pairs involving 61 applications.

□

In this paper, we introduce the *Text-to-Constraints* problem: given a dataset and a set of normative requirements derived from related regulatory text, map those requirements to executable data-quality constraints over the dataset schema. Unlike constraint discovery, which infers constraints from patterns that may hold only incidentally, it requires reasoning over regulatory semantics, schema structure, and violation logic. Like Text-to-SQL [55], it maps natural language to schema-grounded representations, but Text-to-SQL answers specific questions, whereas Text-to-Constraints yields constraints that must hold for all valid instances.

Challenges.

Regulatory documents contain sentences with normative force [16], that is, sentences that prescribe what the data is required to satisfy. The first challenge is mapping these sentences to executable constraints. Example 1 illustrates it: Article 19 states an ordering requirement in prose, whereas checking it requires making explicit the context, attributes, tuple comparisons, and violation condition. More generally, a normative passage may combine conditions, qualifiers, cross-record relationships, and thresholds, while leaving implicit how its terms correspond to elements of the dataset schema.

A second challenge is determining the correct scope of each constraint. Omitting qualifiers yields overly broad constraints that flag valid data, whereas extra conditions yield constraints that are too narrow and miss violations. In Example 1, omitting year/edition or admission category mis-scopes the check.

A third challenge is the choice of the target formalism. Any executable check can be written in SQL, but the same rule admits many syntactically different, equivalent queries, so SQL alone offers no canonical form against which mapped rules can be validated, compared, and reused across documents. More restricted languages are canonical and easy to evaluate, but they miss many cross-record rules, including ordering, quotas, and conditional uniqueness. Example 1 requires comparing applicants within the same context, so we need a canonical constraint language that supports such comparisons and can still be compiled into efficient SQL checks on large datasets (Section 2).

Large language models (LLMs) are a promising way to address these challenges, as demonstrated in related regulatory compliance tasks [17]. However, Text-to-Constraints requires more than identifying relevant passages: the output must be linked to the dataset schema, conform to a constraint formalism, and express violation conditions that can be executed against the data. None of these requirements follows automatically from LLM outputs; they require explicit mechanisms for schema alignment, predicate grounding, and violation semantics.

Other lines of work also bring richer rules or learned components to data quality, but they start from the data. Rule languages such as REEs embed ML predicates in logic rules for data cleaning [6], and LLMs now propose candidate rules from schemas and data samples [8, 22]. Extending DC discovery with richer predicates would still take the data as the only evidence. `Norma` instead takes the rules from the normative text and uses the data only to ground and validate them, through a controlled, schema-derived predicate vocabulary.

From text to data quality constraints.

To address these challenges, we propose `Norma`, the first framework for Text-to-Constraints. Given the sentences of a regulatory text and a target dataset, `Norma` optionally filters the sentences with normative force and maps each of them to a formal constraint over the dataset schema. `Norma` uses denial constraints, which balance expressiveness and control. Unlike Text-to-SQL, it derives reusable constraints with well-defined violation semantics; unlike data profiling, which restricts DCs to few predicates for tractability, it does not search the instance and can therefore use a richer vocabulary with multiple operators, column relationships, null, domain, and pattern predicates, while keeping the output space structured.

`Norma` supports classification and mapping implementations with different cost and quality trade-offs, from single-prompt variants to a structured multi-step mapper. Our experiments show that this framework recovers rules that state-of-the-art data profiling algorithms and Text-to-SQL systems do not capture. `Norma` identifies real compliance failures in publicly released data using constraints mapped from the governing text. A key advantage of this setting is that each detected violation can be traced back to the source passage that justifies the constraint.

A summary of our contributions is the following:

- A formalization of the *Text-to-Constraints* problem;
- `Norma`, the first framework for this problem, with sentence-level classification and mapping implementations with different cost-effectiveness trade-offs, and a compilation of the mapped constraints into SQL violation queries;
- A set of datasets with annotated constraints, and evaluation measures adequate for the Text-to-Constraints problem, including two large real-world datasets from mortgage regulation;

- An extensive empirical study across multiple models, prompting strategies, and baselines from data profiling and Text-to-SQL, showing that Norma recovers critical constraints missed by data-only profiling algorithms.

An extended version of this paper, with additional details and examples, is available with our artifacts [46].

2 PRELIMINARIES

Problem definition. We are given a relational dataset $\mathcal{D}$ and an associated *normative document*. This textual source could be laws, regulations, policy manuals, or operational specifications. It defines the requirements the data must satisfy, expressed in natural language rather than formal, machine-executable constraints.

We model a normative document as a set $\mathcal{S}$ of *segments*, with each segment $s \in \mathcal{S}$ representing a distinct, self-contained textual unit in the document. The segmentation granularity may vary. A segment can correspond, for example, to a single sentence, multiple sentences, or a larger passage. We consider a document normative if it contains at least one segment that expresses a requirement whose compliance can be assessed based on the data.

A *data quality constraint* is a formal rule that $\mathcal{D}$ must satisfy (we discuss the choice of constraint formalism below). Constraints are evaluated on a target relation $r \in \mathcal{D}$ with schema $R = (A_1, \ldots, A_n)$, where each A_i is an attribute with domain $\mathrm{dom}(A_i)$. If a constraint involves multiple relations, we evaluate it on a derived relation obtained by applying the required joins.

Let Φ denote the space of constraints over R. The *Text-to-Constraints* problem is to produce a set of segment–constraint pairs

$$\Pi = \{(s_1, \varphi_1), \ldots, (s_k, \varphi_k)\} \subseteq \mathcal{S} \times \Phi,$$

where $s_i \in \mathcal{S}$ is a segment and $\varphi_i \in \Phi$ is a constraint whose semantics are grounded in the normative content of s_i. Each pair must satisfy two requirements: *executability*, φ_i can be evaluated on any instance of R, and *traceability*, s_i is identifiable as the textual justification for φ_i.

In general, one segment may yield multiple constraints, and a constraint may depend on multiple segments. In this work, we consider sentence-level mapping, where each segment is a single sentence and yields at most one constraint, which may consist of several independent DCs whose violations are united (Section 5.4); coarser granularities, in which constraints are derived by jointly reasoning over several segments, are left for future work.

Denial constraints. We instantiate Φ as the space of *denial constraints* (DCs). A DC specifies that a conjunction of predicates must never be simultaneously true. Formally, given a relation instance r over schema R, a DC φ is a first-order sentence of the form:

$$\varphi : \forall t_1, \ldots, t_m \in r : \neg\big(p_1 \wedge \cdots \wedge p_\ell\big),$$

where each p_i is an atomic predicate over tuple variables and constants, and $m \geq 1$ is the number of tuple variables. While DCs may in principle involve an arbitrary number of tuple variables, we focus on DCs with $m \leq 2$. This restriction is common in the literature and adopted by prior work [14, 37, 44], as DCs involving more than two tuple variables are rare in practice and lead to a dramatic increase in both search and semantic complexity.

Let $A, B \in R$ and let c denote a constant in $\mathrm{dom}(A)$. We allow the following predicate forms: (i) comparisons $u\,\theta\,v$ with $\theta \in \{=, \neq, <, \leq, >, \geq\}$ where u and v are tuple attributes (e.g., $t.A$, $t'.A$, $t.B$), constants, or arithmetic expressions over numeric attributes of the same tuple (e.g., $5 \cdot t.B$); (ii) pattern predicates $t.A$ LIKE p and $t.A$ NOT LIKE p where p is an SQL pattern; (iii) set-membership predicates $t.A$ IN C and $t.A$ NOT IN C where $C \subseteq \mathrm{dom}(A)$; and (iv) null checks $t.A$ IS NULL and $t.A$ IS NOT NULL.

A DC φ is *violated* if there exist tuples for the quantified variables such that all predicates hold simultaneously.

DCs can be translated directly to SQL. For example, the SQL query in Example 1 is actually equivalent to the following DC:

$$\varphi : \forall t, t' \in \mathit{applications} :$$
$$\neg\begin{pmatrix} t.\mathit{year} = t'.\mathit{year} \ \wedge\ t.\mathit{edition} = t'.\mathit{edition} \wedge \\ t.\mathit{institution_id} = t'.\mathit{institution_id} \ \wedge\ t.\mathit{campus_id} = t'.\mathit{campus_id} \wedge \\ t.\mathit{course_code} = t'.\mathit{course_code} \ \wedge\ t.\mathit{shift} = t'.\mathit{shift} \wedge \\ t.\mathit{application_mode} = t'.\mathit{application_mode} \wedge \\ t.\mathit{score} > t'.\mathit{score} \ \wedge\ t.\mathit{ranking} > t'.\mathit{ranking} \end{pmatrix}.$$

Why DCs. DCs provide a compact formalism for a broad class of integrity constraints, including checks, keys, functional dependencies, order dependencies, conditional variants, and other cross-row conditions. They are also directly checkable on a relation through their violation semantics, making them a natural target for compliance-oriented rule extraction. Compared to SQL, which allows many syntactically different but equivalent formulations of the same rule, DCs offer a more uniform and canonical representation, yielding a more stable target space for extraction and evaluation. Finally, with the standard restriction to at most two tuple variables, the space of candidate constraints remains manageable in practice.

3 SYSTEM OVERVIEW

This section presents Norma, our proposed framework for the Text-to-Constraints problem.

Intuitively, one could address this problem with a single LLM prompt that includes the entire document, the dataset schema, and extraction instructions. Such a prompt, however, must jointly locate statements scattered across the text, align them with the schema, and build valid constraints, and it does not link each constraint to the text that justifies it; recovering this link requires an attribution step, such as summarizing related sentences and then mapping each summary.

We therefore decompose the problem into a pipeline that operates at the sentence level, processing each sentence individually. This design provides fine-grained traceability without an attribution step, since it links each constraint to the specific sentence that justifies it, and allows each component of the pipeline to be tuned independently.

Norma takes as input a normative document, segmented into sentences, and a relational dataset and outputs sentence–constraint pairs. Figure 1 outlines the pipeline.

Preprocessing. An automated pipeline converts documents (e.g., PDF files) into normalized text: it extracts the text, removes hyphenation, normalizes whitespace, and segments sentences, followed by minimal manual cleanup. We treat this step as input preparation.

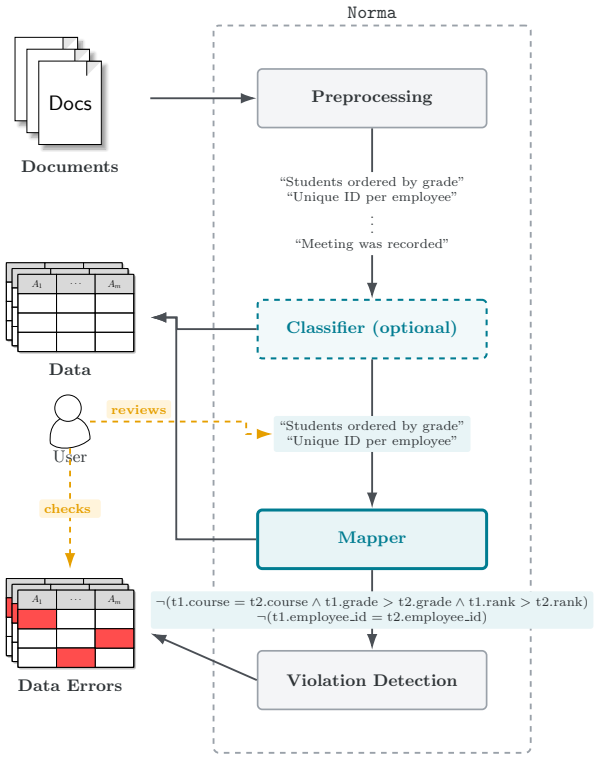


Figure 1: Overview of the Norma pipeline. Blue components represent the novel contributions presented in this paper.

Constraint classifier. Given the sentences from preprocessing, a lightweight classifier can optionally identify those that likely express checkable constraints, filtering out content that is clearly unrelated to data quality constraints. Since constraints missed at this stage cannot be recovered downstream, we prioritize recall at this stage. False positives can still be ruled out by the mapper (which we define next). The classifier only labels the given sentences and never generates text, so it preserves the link between each sentence and its constraint. When no classifier is used, all sentences are passed directly to the mapper. A user can optionally be involved at this stage, reviewing and correcting the set of classified sentences before proceeding to mapping. We describe the classifier details in Section 4.

Constraint mapper. The mapper is the central and most complex component of Norma. It takes each sentence, optionally pre-filtered by the classifier, and produces a denial constraint grounded in the dataset schema. It must interpret the sentence, align it with schema elements, and synthesize a constraint with correct violation semantics. Outputs that fail validation are rejected, and sentences that yield no valid constraint are excluded (Section 5).

Violation detection. Each constraint is validated by compiling it into a SQL violation query and executing it on the target relation (Section 5.4), and it is linked back to the sentence that produced it. Norma returns the violating tuples to the user for further inspection. DC violation detection is a well-studied problem, with specialized systems available for large-scale consistency checks [28, 35, 45]. We compile DCs into SQL because the queries run on any engine that holds the data and also serve the data-based checks of the mapper; specialized engines can replace this step for larger workloads.

4 CONSTRAINT CLASSIFICATION

Given the segment set $\mathcal{S}$ extracted from a regulatory document, the constraint classifier aims at identifying the subset $\mathcal{S}_c \subseteq \mathcal{S}$ of segments that state data quality constraints. A segment is a positive example if it expresses a condition that the data is expected to satisfy and that can be validated against a dataset. Many segments only define terms, describe procedures, or provide context. Constraint-related segments appear less often, and many segments that sound like rules are still too vague to test with data.

For ease of exposition, we treat each segment as a single sentence from now on, unless stated otherwise.

Example 2 (Restaurant inspections). *The NYC Department of Health publishes restaurant inspection results, including scores and letter grades,*[4] *alongside a document describing the grading and inspection regulations.*[5] *Table 1 shows sample sentences from these regulations. The first two state checkable rules (score ranges to grades; inspection frequency per establishment). The last two are not: one is too underspecified to test, and the other describes user permissions rather than a condition on the data.* □

Sentence	*Constraint?*
A score of 0–13 results in a grade of A.	*Yes*
Every food service establishment is scheduled for at least one inspection per year.	*Yes*
Only certain inspections result in a grade.	*No*
Consumers may check an establishment's inspection history.	*No*

Table 1: Constraint vs. non-constraint sample sentences from NYC restaurant inspection regulations.

Norma implements the classifier by prompting an LLM, which can incorporate schema and data samples at inference time and needs no labeled training data.

Constraint Classification by Example

The model labels each sentence as stating a data quality rule or not; Figure 2 shows the prompt structure.

The model returns one label per numbered sentence through a JSON schema. Malformed outputs are retried, then the sentences of the batch are classified one at a time, and a sentence that still has no valid label is marked as non-constraint. Section 6 compares several models and the prompt variants described below.

Prompt structure. Following standard practice in prompt-based text classification [58], we include a task definition, category descriptors, and examples. These elements are essential because constraint classification presents two main challenges. The first challenge is that the positive class is rare and spans many syntactic

[4] Dataset available at https://data.cityofnewyork.us/Health/DOHMH-New-York-City-Restaurant-Inspection-Results/43nn-pn8j. See Section 6 for details.
[5] Available at https://www.nyc.gov/site/doh/services/restaurant-grades.page.

Task Definition	For each sentence, decide whether it states a DATA QUALITY CONSTRAINT that can be validated from the dataset when the required fields are available.
Rule Types	Required/non-null field; Allowed or forbidden values; Numeric bounds; Conditional requirements; Cross-row consistency; Determinacy; Temporal constraints; …
Exclusions	Headings, titles, citations; Definitions or descriptions; Procedures or human decisions; Statements not testable from the dataset; …
Notes	Modal verbs ("may", "should") do not disqualify a sentence; what matters is whether it describes a data condition or verifiable property; …
Examples *(optional)*	YES: "Field X is required." "Two rows with the same Key must not disagree on Field H." … NO: "Field X represents the total amount." …
Context *(optional)*	Table Schema: [Column Names / Types] Table Sample Rows: [Example Tuples]
Input	Sentences: [Numbered List of Sentences] Return one YES/NO label per sentence number

Figure 2: Constraint classification prompt structure (simplified). Colors indicate: ■ positive label clues, ■ negative label clues, ■ data context, and ■ instructions.

forms. To reduce false negatives, we enumerate rule types that the model should treat as constraints. Otherwise, it may miss cases such as functional dependencies that might use specific language. The second challenge is that non-constraint sentences can be easily mistaken for constraints. Definitions, procedures, and contextual statements often use similar wording, mention data-related concepts, or contain keywords that suggest a requirement. To reduce this confusion, we explicitly add notes on exclusions and edge cases.

Examples. Following few-shot in-context learning [11], the prompt includes a few task examples. They are generic templates that remain fixed across datasets rather than domain-specific examples, so the prompt remains portable without labeled data.

Table Context. We evaluated multiple prompt variants. Some ask the classifier to decide from the sentence alone whether it expresses a checkable constraint. Others also provide table context, including table and column information and sample tuples. Schema context helps reject sentences that cannot be checked against the available fields, mainly improving precision, but it lengthens the prompt and makes predictions sensitive to the sample tuples.

Batch processing. To reduce API overhead, each prompt carries a batch of up to 10 sentences identified by numeric indices.

5 CONSTRAINT MAPPING

Let $\mathcal{S}$ denote the sentences passed to the mapper, either all preprocessed sentences or the subset retained by the classifier. For each $s_i \in \mathcal{S}$, the mapper returns a constraint φ_i over the dataset schema, made of one or more DCs of the form $\neg(p_1 \wedge \cdots \wedge p_\ell)$ (Section 5.4), or no output if s_i does not express a constraint or cannot be linked to the schema. The task is to interpret the sentence, identify the relevant schema elements, select predicates that capture the rule, and produce an executable *violation condition*. It is related to Text-to-SQL [60], but Text-to-SQL retrieves satisfying records, whereas our target is a conjunction of predicates that identifies a violation.

Example 3 (Mapping to DC predicates). *The NYC restaurant inspection rule "A score of 0–13 results in a grade of A" has the violation condition* `t.score ≥ 0 ∧ t.score ≤ 13 ∧ t.grade ≠ 'A'`*: any tuple satisfying all three predicates violates the rule.* □

The same rule may be phrased in many ways, often with implicit references to schema elements, literals, or tuple relationships. We therefore rely on LLMs, motivated by their performance in Text-to-SQL [55]. Unrestricted generation, however, is unreliable: LLMs are known to hallucinate column names and use unsupported operators [18, 21, 55], and we observed that they tend to describe valid states rather than violations. Our design follows two principles. First, a *predicate space* derived from the schema serves as a controlled vocabulary: the mapper selects predicates from it or instantiates typed placeholders of predefined templates. Second, the mapper is biased toward *precision*: an incorrect mapping produces false violations, whereas an unmapped sentence can still be inspected manually. We implement two mappers over the same predicate space, a *single-prompt* mapper (Section 5.2) and a *multi-step* mapper (Section 5.3); both produce DCs that are compiled into SQL violation queries (Section 5.4).

5.1 Predicate Space

DC discovery operates over a *predicate space*, the set of predicates allowed in candidate constraints [14, 44]. Because discovery enumerates combinations of predicates, this space is kept small. Regulatory text, however, frequently refers to enumerations, ranges, string formats, and arithmetic conditions. Comparisons with constants and null checks are expressible as DCs, but discovery spaces rarely include set membership, patterns, or arithmetic, because each extension multiplies the candidates to enumerate. `Norma` does not enumerate candidates; it uses the larger space only as a controlled vocabulary, which avoids candidate explosion while preserving syntactic validity.

Predicate templates. We derive the following categories from the schema: (1) *cross-tuple* predicates (e.g., `t1.A = t2.A`), with $\{=, \neq\}$ for type-compatible attributes and also $\{<, \leq, >, \geq\}$ for numeric ones; (2) *same-tuple* predicates (e.g., `t1.A ≤ t1.B`); (3) *null* predicates (`IS [NOT] NULL`); (4) *attribute–constant* predicates (e.g., `t1.A ≥ c`); (5) *pattern* predicates (`t1.A LIKE p`); (6) *set* predicates (`t1.A IN C`); and (7) *arithmetic* predicates over numeric attributes of the same tuple (e.g., `t1.A ≥ c * t1.B`). For categories 4–7, the mapper instantiates c, p, or C with literals from the sentence. Following DC discovery [14], order predicates are generated only for numeric attributes, and two-attribute predicates only for type-compatible pairs with overlapping values.

Coverage and scope. Across our six datasets, 487 of the 490 predicates of the gold DCs belong to this space; the three exceptions, on HMDA Checks, compare an integer-encoded date with arithmetic over the reporting year. The space excludes aggregation and arithmetic over dates or sequences of records, so rules over counts or inspection histories require derived attributes (Section 6.1). The target is a DC with at most two tuple variables and universal violation

semantics: a single violating pair suffices. Rules that must hold only for most pairs, or that compare distances between values, such as differential dependencies, require counting or tolerance semantics and are outside our scope.

Schema-driven, on-demand generation. With arithmetic and code-set templates, the space of a 135-column table reaches hundreds of thousands of templates, most irrelevant to any given sentence. `Norma` therefore stores only a *schema description*: for each column, its type, admitted operators, domain values (when it has few distinct values), and a short description written offline by an LLM from a deterministic data profile, merged with the official data dictionary when available (HMDAZ). Once the relevant columns of a sentence are selected, predicates are generated only for them. For a fixed instance, the generated set equals the full space restricted to those columns, so the vocabulary is unchanged; only its materialization is deferred.

5.2 Single-Prompt Mapper

The single-prompt mapper performs the full mapping in one LLM call for a batch of three sentences. The prompt contains the definition of a DC, a small set of examples covering thresholds, set membership, field comparisons, nulls, cross-tuple rules, and non-constraints, and the columns retrieved for the batch: columns are ranked by the embedding similarity between the sentence and the column name and description, and the prompt carries the description, domain values, and generated predicates of each retrieved column. For each sentence, the model returns a list of independent rules, whose violations are united (Section 5.4), or `null` if the sentence is not mappable.

Violation form. Without explicit guidance, LLMs tend to output predicates describing *satisfying* tuples rather than *violating* ones, since regulatory sentences describe what must hold. In Example 3, the correct output is `grade ≠ 'A'`, not `grade = 'A'`. The prompt states the criterion explicitly ("must be X" ⇒ `!= X`; "must not be X" ⇒ `= X`), and the model must first state the condition under which a tuple violates the rule (`violated_when`) before writing the DC.

Cross-tuple constraints. For rules that relate records, the mapper must introduce two tuple aliases and a join on the shared identifier; cues such as "for the same" or "repeated" in the prompt suffice for the model to generalize to other phrasings.

Literal grounding. A recurring failure is to produce thresholds, strings, or enumerations in the wrong format or casing. The domain values of each retrieved column anchor literals to the data representation (e.g., a label such as `'Secured by a first lien'` rather than a paraphrase).

Validation. Every predicate must reference existing columns, use admitted operators and typed constants, and have no placeholders. Malformed outputs are retried, invalid rules are returned once to the model with the error, and a sentence whose attempts all fail produces no constraint. Membership in the predicate space is not enforced; we report out-of-space predicates instead.

5.3 Multi-Step Mapper

A single prompt must solve schema linking, predicate selection, literal grounding, and the violation form jointly, so early errors propagate: a wrong column breaks predicate selection, a hallucinated literal yields a valid but meaningless DC, and a non-constraint triggers an unnecessary mapping. In Text-to-SQL, separating schema linking from later decisions often improves accuracy [18, 21, 30, 52]. The multi-step mapper uses four LLM calls, each receiving the output of the previous one (Figure 3), followed by deterministic checks. Each stage isolates one decision that the single-prompt mapper makes jointly, trading additional calls for smaller decisions that can each be validated. In our experiments, this decomposition keeps the mapper usable with backbones whose single-prompt outputs fail (Section 6.6); a finer ablation that removes or reorders individual stages is left for future work.

Step	Context	Task
Schema	Schema description	Identify all columns relevant to *s*: direct mentions, synonyms, and derived concepts. Abort if no concept maps to a column.
Predicates	↪ Selected columns and their predicates	Select predicates and templates and the constraint pattern (value mapping, temporal bound, dependency, required field, conditional rule). Abort if none fits.
Rule filter	↪ Columns and predicates	Decide whether *s* is a specific rule verifiable from column values alone. Reject: processes, permissions, vague quantifiers.
Construction	↪ Selected predicates, column profiles	Instantiate placeholders with literals from *s* and return the DC in violation form.

Figure 3: Prompt structure of the multi-step mapper; context narrows progressively. ■ input context; ■ abort / reject signal; ■ instruction.

Stages. *Schema linking* identifies the columns relevant to the sentence, including implicit references, and rejects the sentence if none applies. *Predicate selection* chooses, among the predicates generated for those columns, the ones that together form the DC pattern; a dedicated structural step improves accuracy in Text-to-SQL [30]. The *rule filter* decides whether the sentence maps to a DC under the grounding obtained so far. Unlike the classifier (Section 4), it sees the candidate columns and predicates, and many sentences that look like rules in isolation support no executable constraint once grounded. Placing it before construction avoids a call for rejected sentences. *Construction* instantiates the selected templates with thresholds, constants, patterns, and enumerations aligned with the data representation. Framing this step as grounded instantiation rather than open-ended generation makes unsupported predicates and malformed expressions less likely [33, 49, 53].

Data-grounded checks. Each candidate DC is first validated structurally: predicates must belong to the predicate space, reference valid columns, use type-compatible operators, and contain no placeholders. Next, for each equality, string inequality, `LIKE`, or membership predicate with literals, we issue an existence query; a predicate that no tuple satisfies is an *orphan predicate*, and a DC containing

one is discarded. Numeric bounds and null predicates are exempt, since their validity is structural. Finally, we execute the DC and discard it when more than 90% of the tuples take part in a violation: a DC that flags almost every tuple most likely encodes the valid state rather than its violation.

5.4 From Denial Constraints to SQL

Every DC is compiled into a SQL *violation query*, used by the data-grounded checks, for violation detection, and for evaluation. Compilation is deterministic, but it settles questions that the DC formalism leaves open on real data.

Rule structure and compilation. Each rule is a tree of `AND`/`OR` nodes over the predicate forms of Section 2; a rule with disjunctions is equivalent to one DC per disjunct of its disjunctive normal form. Independent rules of a sentence are compiled separately, and their violations are united. A rule with one tuple variable becomes a selection returning the identifiers of the violating tuples; a rule with two becomes a self-join with $t_1.tid \neq t_2.tid$. Membership and pattern predicates map to `IN` and `LIKE`, division is guarded against zero, and literals are canonicalized. The self-join condition must not be replaced by $t_1.tid < t_2.tid$, a common way to avoid reporting a pair twice: this is only correct for symmetric DCs, such as FDs. For the asymmetric ordering rule of Example 1, it misses every pair whose witnesses appear in the opposite order (61 applications flagged with $\neq$, only 9 with $<$). Since violations are reported per tuple, enumerating both orders does not inflate the counts.

Types, missing values, and execution. Published data rarely matches its declared schema; numeric fields contain codes such as `NA` or `Exempt`. We read the data as text, type each column with the schema, and compare numeric attributes through `TRY_CAST`, which turns malformed values into `NULL` instead of aborting the query. Under three-valued logic, a predicate over `NULL` is unknown and the tuple is not reported, so requiredness must be stated with `IS NULL`, and `loan_type != 1` silently skips tuples without a loan type; the prompts illustrate this explicitly. Queries run on DuckDB under a memory budget, a time limit, and a result cap, since a pair DC that joins on a low-cardinality attribute can produce billions of bindings. A query that exceeds them is reported as an execution error, never as a constraint without violations.

Bridging text and data. Most of the difficulty lies before compilation, in the inferences that make the query check what the text means (Table 2). On Tax, "lower salary cannot have higher tax rate" becomes an asymmetric pair DC. On SISU, "selected in only one of their options" requires identifying the student by the national ID rather than the exam registration number, which the text leaves implicit. On HMDA Checks, amounts are recorded in thousands, so "$1,000 ($1 million)" becomes `loan_amount >= 1000`, and "five times the income" becomes an arithmetic predicate. On HMDAZ, the regulation compares the APR with the average prime offer rate, a difference stored as `rate_spread`, and combines coverage conditions, alternative thresholds, and labels that must match the data verbatim. Each decision changes the violating tuples, which is why we report violation detection next to structural extraction.

5.5 Methodological Contributions

Table 3 delineates which components of `Norma` build on Text-to-SQL practice and which are specific to mapping normative text to constraints. Prompts implement individual decisions, but replacing a prompt changes neither the vocabulary, the violation form, nor the checks that every constraint must pass.

6 EXPERIMENTAL EVALUATION

We implemented a prototype of `Norma` and evaluated it on several datasets prepared for the *Text-to-Constraints* setting. Our evaluation addresses four questions: **RQ1**, how accurately can `Norma` identify the sentences that express data constraints? **RQ2**, how accurately can it map them to executable DCs that capture the corresponding violations? **RQ3**, how do classifier and mapper implementations, LLM backbones, and prompt variants affect quality and robustness? **RQ4**, how does `Norma` compare with data profiling and Text-to-SQL baselines?

6.1 Datasets

We evaluate `Norma` on six datasets: four based on real-world regulatory documents paired with corresponding tabular datasets, and two derived from benchmark datasets commonly used in data quality and profiling research. Table 4 summarizes them. For every dataset, the authors labeled each sentence and wrote its gold DC in violation form over the released schema; sentences not checkable on the released attributes are non-constraints. No sentence occurs twice, and near-identical sentences (similarity ≥ 0.9) are templated statements that differ in one attribute or literal (13 pairs in SISU, 32 in HMDA Checks, at most two elsewhere), never sharing a gold DC.

SISU. This dataset contains admission records of Brazil's university selection system (Example 1). We annotated three sections of *Portaria Normativa No. 21/2012* covering student registration, candidate classification and selection, and waitlist procedures. The constraints span 11 nullability checks, a functional dependency, an order dependency (Example 1), a conditional functional dependency (a student is selected in at most one option), and three conditional DCs over one or two tuples. Many have violations in the released data.

Restaurant. This dataset pairs the NYC inspection records (Example 2) with the grading and closure sections of the food service inspection guide, which constrain score-to-grade mappings, inspection frequency, grade posting, and closure conditions. For the one rule over inspection sequences, we added a derived column with the days until the next inspection; automating such transformations is future work.

Tax and Hospital. Both datasets are standard benchmarks in DC research [14, 51], but their constraints exist only in formal form. An LLM (Claude 4.5 Sonnet, not evaluated in any `Norma` configuration) wrote a description of each constraint and surrounding non-constraint sentences in schema terminology, mimicking a document that interleaves rules with other prose. These two datasets nevertheless form a controlled setting: their constraints come from prior papers and their sentences from an LLM that may share priors with the evaluated backbones, which likely makes them easier

Dataset	Sentence (abridged)	Violation condition in the compiled query (`WHERE` clause, casts omitted)	Viol.
Tax	Within the same state, a person with lower salary cannot have higher tax rate.	`t1.tid <> t2.tid AND t1.state = t2.state AND t1.salary < t2.salary AND t1.rate > t2.rate`	363
SISU	The student will be selected in only one of their options.†	`t1.tid <> t2.tid AND t1.cpf = t2.cpf AND t1.edicao = t2.edicao AND t1.ano = t2.ano AND t1.etapa = t2.etapa AND t1.opcao != t2.opcao AND t1.aprovado = 'S' AND t2.aprovado = 'S'`	149
HMDA Checks	If loan amount and income are > 0 and the loan amount is ≥ $1,000 ($1 million), then loan amount should be < 5 times the income.	`t1.loan_amount > 0 AND t1.applicant_income > 0 AND t1.loan_amount >= 1000 AND t1.loan_amount >= (5 * t1.applicant_income)`	600
HMDAZ	A high-cost mortgage is a consumer credit transaction secured by the consumer's principal dwelling in which the APR exceeds the APOR by more than (A) 6.5 points for a first lien, (B) 8.5 points for a first lien on personal property under $50,000, or (C) 8.5 points for a subordinate lien.	`t1.business_or_commercial_purpose = 2 AND t1.occupancy_type_name = 'Principal residence' AND t1.reverse_mortgage_name = 'Not a reverse mortgage' AND t1.hoepa_status_name = 'Not a HOEPA loan' AND ((t1.lien_status_name = 'Secured by a first lien' AND (t1.manufactured_home_secured_property_type != 2 OR t1.loan_amount >= 50000) AND t1.rate_spread > 6.5) OR (t1.lien_status_name = 'Secured by a first lien' AND t1.manufactured_home_secured_property_type = 2 AND t1.loan_amount < 50000 AND t1.rate_spread > 8.5) OR (t1.lien_status_name = 'Secured by a subordinate lien' AND t1.rate_spread > 8.5))`	4

Table 2: Gold constraints compiled into violation queries, with the number of violating tuples (HMDA: first 1M rows). Numeric attributes are compared through `TRY_CAST`. †Translated from Portuguese.

Component	Origin	What is specific to `Norma`
Schema linking	Adapted [30, 49]	Over a schema description with column descriptions and domain values.
Staged decomposition	Adapted [49]	A rule filter decides whether the text is a checkable rule at all; every Text-to-SQL question has an answer.
Predicate space	New	Typed templates derived from the schema and generated on demand.
Violation form	New	The output is the condition that must never hold (a DC), a canonical target instead of a query.
Data-grounded checks	New	Orphan literals and the share of violating tuples are checked on the instance.
Compilation	New	Pair, type, and `NULL` semantics of violation queries.
Evaluation	New	Equivalence-aware extraction, execution equivalence, per-rule metrics (Section 6.3).

Table 3: Components adapted from Text-to-SQL and components specific to Text-to-Constraints.

Dataset	#Rows	#Cols	Sent.	Const.	Viol.
Hospital	1k	17	109	15	834
Restaurant	128k	21	33	7	52,311
SISU	102k	56	42	17	533
Tax	100k	15	126	6	468
HMDAZ†	1M	135	17	9	209
HMDA Checks†	1M	81	158	92	986,491

Table 4: Dataset statistics. †First 1M rows of the public file (15.1M and 4.8M rows); *Const.* counts sentences mapped to a non-null gold DC.

than real regulatory text. SISU, Restaurant, and the HMDA datasets, which pair real regulatory text with real data, are the more reliable signal.

HMDAZ and HMDA Checks. Both datasets use public loan-level records released under the Home Mortgage Disclosure Act (HMDA); we use the first 1M rows of each file. *HMDAZ* pairs the 2018 loan application register (135 columns, including the code and label versions of each categorical field) with Sections 1026.32, 1026.35, and 1026.43 of Regulation Z, which define high-cost (HOEPA) and higher-priced mortgages, the limitations that apply to them, and the ability-to-repay requirements. Sentences are verbatim from the regulation, and gold DCs keep the coverage conditions of each trigger. Scope-only text, or text whose outcome is not observable in the data, is annotated with a `null` constraint. *HMDA Checks* pairs the 2014 register (81 columns) with the 2014 HMDA edit specification published by the FFIEC: one entry per edit, with the edit test verbatim; edits that cannot be checked on a single record (aggregate, transmittal-sheet, or external-reference edits) are `null`. Compared with the other datasets, these two add wide schemas whose relevant column is often named after a code rather than the concept in the text, rules with arithmetic, and highly concentrated violations.

6.2 Baselines

Since no prior system addresses the Text-to-Constraints task directly, we compare `Norma` against the two closest paradigms: DC discovery from data alone and Text-to-SQL.

Data profiling. We use the DC discovery algorithms from [47], running both exact and approximate denial-constraint discovery. Exact DCs hold with no violations and thus cannot match rules violated in the data, so we focus on approximate DCs, which tolerate a fraction ε of violating pairs. Since FDs and order dependencies with at most two tuples are special cases of DCs, approximate DC discovery over the same attributes subsumes their discovery, and we do not run separate FD or OD algorithms. Conditional variants require predicates with constants, which DC discovery supports only for a restricted set of values; a comparison with dedicated CFD discovery is left for future work.

We run exact discovery and $\varepsilon \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$, rank the DCs by coverage and succinctness [14], and report, for each metric, the best result over this grid and top-$k \in \{5, 10, 20, 50, 100\}$.

Since discovery does not scale to the full SISU, Restaurant, and HMDA schemas, we restrict them to the columns of the gold constraints, the most favorable setting for the baseline, and run it on a seeded sample of 100k of the 1M HMDA rows. On HMDA Checks

(39 columns), no configuration finished: runs timed out or exhausted a 12 GB heap even with samples of 10k rows.

Text-to-SQL. We prompt XiYan-SQL [34], a state-of-the-art Text-to-SQL framework, to return for each sentence a query that selects the violating tuples, or `NULL` if the sentence is not a checkable constraint, so it acts as both classifier and mapper. We use the 32B model with three seeds and report the mean.

6.3 Evaluation Metrics

We evaluate three dimensions; as in Text-to-SQL, execution-based evaluation complements structural matching, because exact form alone does not capture correctness [63]. *Constraint classification* measures precision, recall, and F_1 of the sentences labeled as constraints; it applies to `Norma` and Text-to-SQL, not to profiling. *Constraint extraction* (CE) compares predicted and gold DCs by exact predicate-set match (precision, recall, F_1). For Text-to-SQL, predicates are read from the `WHERE` clause (a correlated `EXISTS` as a pair DC); a query with no DC reading, e.g., with aggregation or `CASE`, counts as correct if it returns exactly the violations of the gold DC. *Violation detection* executes all predicted and all gold constraints and compares the sets of tuples that take part in at least one violation (aggregate violation F_1, AV); it applies to all methods.

Complementary mapping metrics. Exact matching penalizes logically equivalent rewritings, and the tuple-level aggregate can be dominated by a single rule. We therefore also report: (i) *equivalence-aware extraction* ($CE_{\equiv}$), which compares DCs after a logical normal form (e.g., `IN` as a disjunction, sorted operands) and unifies literals on a code column and its label column (e.g., `hoepa_status = 1` and `hoepa_status_name = 'High-cost mortgage'`) only when the data confirms that they select the same tuples; (ii) *execution equivalence*, the fraction of gold rules with violations whose predicted DC flags exactly the same tuples; (iii) *per-rule violation* F_1, averaged over gold sentences so that every rule weighs the same; and (iv) *violation concentration*, the share of gold-violating tuples flagged by the largest gold rule. Predicted DCs whose SQL fails are counted as errors, not as producing no violation.

6.4 Implementation and Settings

`Norma` is implemented in Python with DuckDB as the execution backend. All LLM calls go through OpenRouter with strict JSON-schema outputs. We evaluate six backbones: GPT-6 Luna, DeepSeek-V4-Flash, Mistral Large 4, Qwen3.8-2.4T-A95B, GPT-6 Sol, and Claude Sonnet 5.5. Two candidates were replaced: Kimi K2.6, which emitted whitespace until the token limit under strict decoding, and MiniMax-M3, which returned empty single-prompt outputs (Section 6.6). Classifier and mapper configurations run once, with up to 32k output tokens per call; end-to-end configurations run three times. A preliminary study with earlier backbones (e.g., Qwen3-32B and GPT-5) followed the same protocol on Hospital, Restaurant, SISU, and Tax; we refer to it in Section 6.6. `Norma` and DC discovery ran on a workstation with an Intel Core i7-7700HQ CPU and 16 GB of memory, and XiYan-SQL on an NVIDIA A100 (80 GB). The baselines are scored by the same evaluator, data, and query limits as `Norma`.

6.5 Classifier Performance

Our first experiment evaluates how model and prompt design affect constraint classification. We derive four variants of the prompt in Figure 2 by enabling or disabling *examples* and *table context* (*No Context*, *Table Context*, *Examples*, *Examples + Table Context*) and evaluate them with six LLMs on all datasets (Figure 4). Because sentences missed here cannot be recovered by the mapper, we require recall ≥ 0.90 on every dataset when selecting a default configuration.

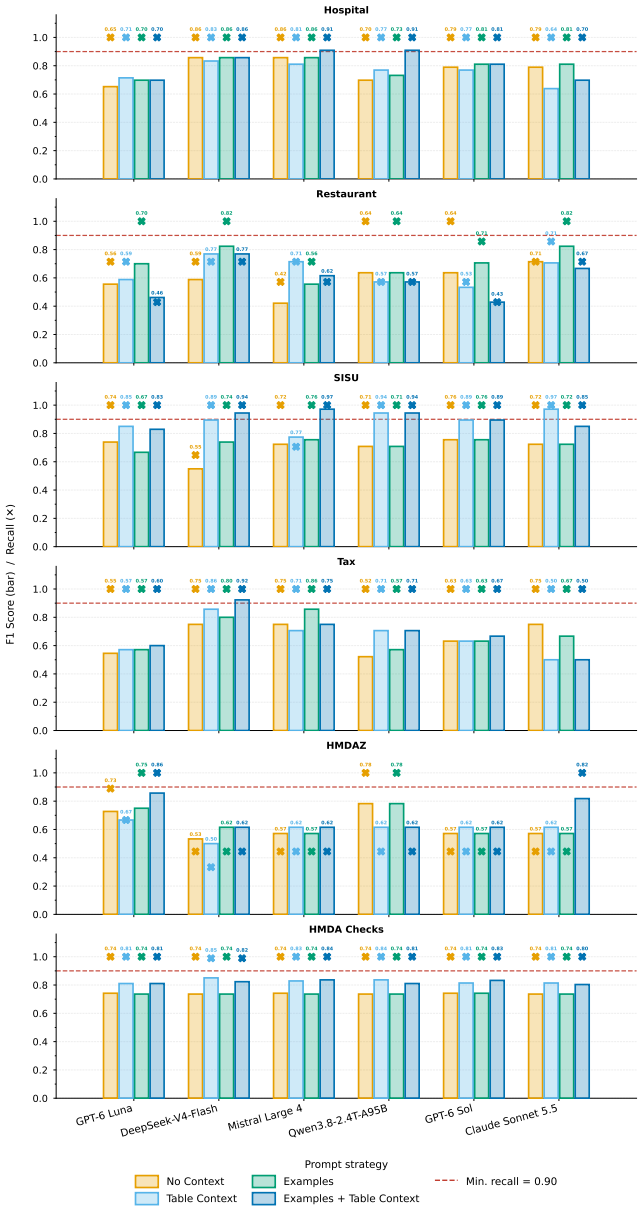


Figure 4: Constraint classification F_1 (bars) and recall (x) across models and prompt variants. The dashed line marks the 0.90 recall threshold.

On Hospital, Tax, SISU, and HMDA Checks, almost every configuration reaches recall 1.00, so differences lie in precision. On HMDAZ and Restaurant, recall is the bottleneck: on HMDAZ, most configurations recover only 4 of the 9 constraint sentences, missing regulatory definitions (e.g., when a mortgage is high-cost) that read as scope statements, although each implies a checkable condition. Only three configurations satisfy this recall requirement on all six datasets: *Qwen3.8-2.4T-A95B* with *Examples* (mean F_1 0.69), *GPT-6 Luna* with *Examples* (0.69), and *Qwen3.8-2.4T-A95B* with *No Context* (0.68); we select the first. Configurations with higher mean F_1 exist (e.g., DeepSeek-V4-Flash with *Examples + Table Context*, 0.82), but they miss more than half of the HMDAZ constraints.

Table context has two opposite effects. It is the only signal that raises precision on HMDA Checks, where non-constraint edits are phrased like checkable ones but refer to aggregates or external data: without it, F_1 equals that of the all-positive classifier (0.74); with it, 0.80–0.85. On wide schemas, however, it does not help recall: on HMDAZ (135 columns), 9 of the 12 table-context configurations stay at or below 0.44. Averaged over models and datasets, *Examples* gives the best recall (0.93), and table context higher precision (0.71 vs. 0.58–0.62) at lower recall (0.85–0.87). Without examples or context, classification is already strong on the controlled datasets (recall 1.00, mean F_1 0.72 on Tax and Hospital), and the best variant adds only 0.04. On SISU, Restaurant, and the HMDA datasets, the gain is larger (0.66 to 0.75 with *Examples + Table Context*), mostly through precision. A plausible explanation is that LLM-written sentences resemble text these models produce or saw in pretraining; we cannot verify such exposure, but context matters most on real regulatory text.

6.6 Constraint Mapping Performance

Our second experiment evaluates how model and mapper choices affect mapping quality. To isolate mapping from classification errors, we compute constraint extraction and aggregate violation F_1 on the ground-truth constraint sentences only, comparing the *Single-Prompt* and *Multi-Step* mappers with six LLMs (Figure 5, Table 5). On Hospital, Restaurant, SISU, and Tax, *Multi-Step* improves both metrics (mean over models: 0.81 vs. 0.75 and 0.83 vs. 0.55), with the largest gains on Restaurant and SISU, whose rules combine several conditions or relate two tuples. The best Multi-Step configurations exceed the best one of the preliminary study (GPT-5, 0.78 and 0.95): Claude Sonnet 5.5 reaches 0.85 and 0.97, GPT-6 Luna 0.82 and 0.98, and Qwen3.8-2.4T-A95B 0.80 and 1.00. Decomposition also keeps the mapper usable with unstable backbones: MiniMax-M3 returned empty outputs for 107 of 156 Single-Prompt sentences but completed the Multi-Step pipeline without failures; among the final backbones, only DeepSeek-V4-Flash has output failures (3 Single-Prompt sentences).

Where Single-Prompt loses violations. On Restaurant, one rule (at least one inspection per year) flags 99% of the gold-violating tuples. Every Single-Prompt configuration except Qwen3.8-2.4T-A95B returns `null` for it, so aggregate recall falls to 0.01; without this rule, all reach 1.00. On SISU, Single-Prompt identifies the student by the exam registration number instead of the national ID in the two-tuple rules, flagging about 51k tuples instead of 533.

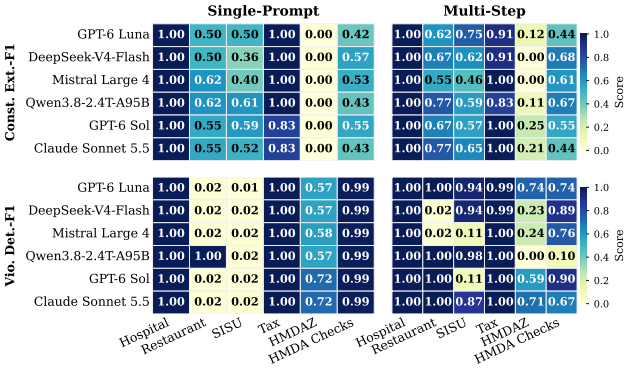


Figure 5: Sentence-level mapping results across models and mappers: constraint extraction F_1 and aggregate violation F_1 on ground-truth sentences.

Multi-Step, which links columns and selects predicates before writing the DC, avoids both errors with most backbones, but not all: DeepSeek-V4-Flash and Mistral Large 4 also miss the Restaurant rule, and GPT-6 Sol and Mistral Large 4 make the identification error in one SISU rule (8.9k tuples; aggregate 0.11).

Mapper	Model	CE	$CE_{\equiv}$	AV	Rule-F_1	Exec.	Fail.
Single-Prompt	GPT-6 Luna	0.57	0.69	0.60	0.78	0.75	0
	DeepSeek-V4-Flash	0.57	0.66	0.60	0.74	0.70	3
	Mistral Large 4	0.59	0.66	0.60	0.73	0.71	0
	Qwen3.8-2.4T-A95B	0.61	0.70	0.76	**0.86**	**0.83**	0
	GPT-6 Sol	0.59	0.67	0.62	0.76	0.72	0
	Claude Sonnet 5.5	0.55	0.64	0.62	0.76	0.70	0
Multi-Step	GPT-6 Luna	0.64	0.69	**0.90**	0.77	0.76	0
	DeepSeek-V4-Flash	0.65	0.69	0.68	0.68	0.69	0
	Mistral Large 4	0.60	0.64	0.52	0.74	0.73	0
	Qwen3.8-2.4T-A95B	0.66	0.68	0.68	0.81	0.77	0
	GPT-6 Sol	0.67	0.71	0.77	0.79	0.77	0
	Claude Sonnet 5.5	**0.68**	**0.74**	0.88	0.84	0.82	0

Table 5: Mean mapping quality over the six datasets (ground-truth sentences). CE: exact extraction F_1; $CE_{\equiv}$: equivalence-aware extraction F_1; AV: aggregate violation F_1; Rule-F_1: mean per-rule violation F_1; Exec.: execution equivalence; Fail.: sentences with no valid output.

HMDA datasets. Exact extraction stays low (at most 0.25 on HMDAZ and 0.68 on HMDA Checks). Many predicted DCs are logically equivalent to the gold ones but written differently (`IN` instead of a disjunction, a label instead of its code column); others omit the coverage conditions of a trigger. Equivalence-aware extraction recovers much of the first group: on the HMDA datasets it raises mean extraction F_1 from 0.34 to 0.46 for Multi-Step and from 0.24 to 0.51 for Single-Prompt, while it barely changes elsewhere. On HMDA Checks, aggregate violation is not informative: two date-range edits flag 99% of the gold-violating tuples, which cover 98.6% of the rows, so a DC flagging every tuple already scores 0.99. Single-Prompt reaches 0.99 with every backbone, mostly through such DCs. Multi-Step scores 0.67–0.90 with five backbones, but only GPT-6 Sol

maps a date edit correctly; the others write incorrect DCs for other edits that flag 480k–550k rows and pass the 90% violation-share check. Without the two date edits, the Multi-Step aggregate falls to 0.12–0.19 for four backbones and stays at 0.98–1.00 for GPT-6 Sol and Qwen3.8-2.4T-A95B, while Single-Prompt ranges from 0.18 to 1.00. Per-rule metrics are far less sensitive: mean per-rule F_1 is 0.78 (Multi-Step) and 0.86 (Single-Prompt) with all edits, and 0.84 and 0.85 without the date edits.

Default mapper and sensitivity to the backbone. With Multi-Step, Claude Sonnet 5.5 has the best mean extraction (0.68) and GPT-6 Luna the best mean aggregate violation (0.90). The mean of the two selects *Claude Sonnet 5.5* (0.776, against 0.771 for GPT-6 Luna and 0.720 for GPT-6 Sol), a tie with GPT-6 Luna given single runs. Backbones differ more in violation detection than in extraction: mean exact extraction ranges only from 0.60 to 0.68, aggregate violation from 0.52 to 0.90, and per-rule metrics lie in between (0.68–0.84). The divergences concentrate on implicit decisions rather than syntax: which attribute identifies an entity (SISU), whether a definition implies a checkable trigger (HMDAZ), and how integer-encoded dates are compared (HMDA Checks). Single runs also vary: two runs of Mistral Large 4 on Restaurant, differing only in a check unrelated to the dominant rule, scored 1.00 and 0.02 in aggregate because that rule was mapped in one run only. We cannot relate these differences to model size or training data, which most backbones do not disclose.

6.7 End-to-End Evaluation and Comparison

We evaluate the full pipeline with the default components, *Qwen3.8-2.4T-A95B* with *Examples* as classifier and *Claude Sonnet 5.5* with *Multi-Step* as mapper, in two variants: classifier plus mapper (C+MS) and the mapper alone, which relies on its rule filter (Multi-Step). Each variant runs three times, and Table 6 reports the mean. The baselines are run on all six datasets as described in Section 6.2.

`Norma` is the only approach that produces gold constraints on every dataset. Extraction F_1 is 1.00 on Hospital and Tax, 0.67 on Restaurant, 0.65 on SISU, 0.44–0.45 on HMDA Checks, and 0.14–0.25 on HMDAZ (0.35 with equivalence-aware matching), against 0.00–0.80 for XiYan-SQL and zero for DC discovery. Classification F_1 of C+MS lies between 0.79 and 0.91, above XiYan-SQL (0.55–0.86) on every dataset, including the wide HMDA schemas.

The classifier mainly improves precision: compared with Multi-Step alone, C+MS raises classification precision on Hospital (0.81 vs. 0.48), Restaurant (0.89 vs. 0.65), and Tax (0.71 vs. 0.33). On Tax, this decides the aggregate violation score. Multi-Step alone maps about 12 non-constraint sentences per run, written to resemble constraints; one turns "major cities exist within fixed states" into `t1.city = t2.city AND t1.state != t2.state`, which flags 52k tuples, so the aggregate falls to 0.02. C+MS filters most of them and reaches 0.70 on average (0.55–1.00). On the HMDA datasets, both variants make almost the same decisions; on HMDAZ, they still write different DCs for the same triggers (exact extraction 0.14 vs. 0.25, equivalence-aware 0.35 for both), and the aggregate varies from 0.02 to 0.75 across runs.

The violation results expose two further failure modes. First, a broad DC on a non-constraint sentence can dominate the aggregate: on SISU, both variants map sentences that define quota categories, labeled as non-constraints, to checks that flag up to 12k tuples, so the aggregate stays at 0.05–0.06, close to XiYan-SQL (0.08), although the gold rules are recovered well (per-rule violation F_1 0.91–0.95, execution equivalence 0.86–0.90). Second, missing a dominant rule: on Restaurant, the yearly-inspection rule flags 99% of the gold violations, and Multi-Step alone returns no DC for it in all three runs, C+MS in two of them, because the rule filter judges this cross-year requirement as not expressible, although the derived attribute encodes it. The aggregate is therefore 0.02 for Multi-Step and 0.35 for C+MS, below DC discovery (0.58) and XiYan-SQL (0.52). Both failures depend on the backbone and the run: with GPT-6 Sol as the mapper, the pipeline reached 1.00 on Restaurant and up to 0.98 on SISU, but 0.15 on Tax. On HMDA Checks, neither the aggregate of 0.71 nor XiYan-SQL's 0.99 is evidence of quality: 98.6% of the rows violate some gold rule, so broad queries score high; `Norma` maps no date-range edit, and its score comes from an incorrect DC that flags 481k tuples. There, 83% of the gold rules with violations are reproduced exactly, and the mean per-rule F_1 is 0.77–0.78.

Family	Method	Constr. Classification			Constr. Extraction			Viol. Detection		
		P	R	F_1	P	R	F_1	P	R	F_1
Hospital										
`Norma`	C+MS	0.81	1.00	**0.89**	1.00	1.00	**1.00**	1.00	1.00	**1.00**
	Multi-Step	0.48	1.00	0.65	1.00	1.00	**1.00**	1.00	1.00	**1.00**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.83	1.00	0.91
Text2SQL	XiYan-SQL	0.51	1.00	0.68	0.66	1.00	0.80	0.83	1.00	0.91
Restaurant										
`Norma`	C+MS	0.89	0.71	**0.79**	0.80	0.57	**0.67**	1.00	0.34	0.35
	Multi-Step	0.65	0.71	0.68	0.80	0.57	**0.67**	0.87	0.01	0.02
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.41	1.00	**0.58**
Text2SQL	XiYan-SQL	0.71	0.71	0.71	0.50	0.43	0.46	0.44	0.64	0.52
SISU										
`Norma`	C+MS	0.67	1.00	**0.80**	0.65	0.65	**0.65**	0.03	0.98	0.06
	Multi-Step	0.65	1.00	0.79	0.65	0.65	**0.65**	0.03	0.97	0.05
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.01	1.00	0.01
Text2SQL	XiYan-SQL	0.92	0.65	0.76	0.42	0.29	0.35	0.07	0.76	**0.08**
Tax										
`Norma`	C+MS	0.71	1.00	**0.82**	1.00	1.00	**1.00**	0.59	1.00	**0.70**
	Multi-Step	0.33	1.00	0.50	1.00	1.00	**1.00**	0.01	1.00	0.02
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.00	1.00	0.01
Text2SQL	XiYan-SQL	0.38	1.00	0.55	0.33	0.83	0.48	0.01	1.00	0.01
HMDAZ										
`Norma`	C+MS	0.86	0.89	0.87	0.13	0.15	0.14	0.26	1.00	0.38
	Multi-Step	0.96	0.89	**0.92**	0.25	0.26	**0.25**	0.48	1.00	**0.64**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.34	0.25	0.29
Text2SQL	XiYan-SQL	0.88	0.78	0.82	0.00	0.00	0.00	0.00	0.37	0.01
HMDA Checks										
`Norma`	C+MS	0.90	0.93	0.91	0.45	0.43	0.44	0.99	0.56	0.71
	Multi-Step	0.90	0.93	**0.92**	0.45	0.44	**0.45**	0.99	0.56	0.71
Profiling	DC Discovery	n/a	n/a	n/a	DNF	DNF	DNF	DNF	DNF	DNF
Text2SQL	XiYan-SQL	0.76	1.00	0.86	0.34	0.37	0.36	0.99	1.00	**0.99**

Table 6: End-to-end comparison of `Norma` (C+MS and Multi-Step; mean of three runs) and the baselines. Profiling: DC discovery from data alone; Text2SQL: XiYan-SQL adapted to generate violation queries; DNF, did not finish within the time and memory limits; n/a, metric not applicable; bold, best F_1 per metric and dataset.

The baselines are competitive only on narrower parts of the experiments. DC discovery recovers violation signals when the data distribution strongly reflects the target rules, as on Hospital and Restaurant, but never the gold constraints (extraction F_1 zero on all datasets), and it does not finish on HMDA Checks. XiYan-SQL is competitive on Hospital (extraction 0.80), but on Tax and HMDAZ it returns overly broad queries, with near-zero violation precision. Since a single gold rule flags most gold-violating tuples in four of the six datasets, broad violation sets can still overlap strongly with the gold ones, while a correct pipeline can be penalized by one broad false positive; we therefore recommend reading the aggregate together with the per-rule metrics. Overall, `Norma` is ahead of both baselines in classification and extraction on all six datasets; in violation detection, it is ahead on Hospital, Tax, and HMDAZ and behind on Restaurant, SISU (by 0.02), and HMDA Checks.

Usability and cost. The returned set stays small and reviewable: C+MS returns 6–25 DCs per run on Hospital, Restaurant, SISU, and Tax, 9 on HMDAZ, and 95 on HMDA Checks, close to the number of gold constraints (6–92). Users can rank the DCs by the number of tuples they flag, read the source sentence of each, and use the optional review step (Section 3); the broad DCs above stand out precisely by their violation counts. All reported experiments cost US$166 through OpenRouter (classifiers US$5.7, mappers US$52.9, three end-to-end runs US$107.2). With Claude Sonnet 5.5, an end-to-end run costs US$0.4–2.9 on Hospital, Restaurant, SISU, and Tax and about US$9.7 on HMDA Checks (158 sentences, 1M rows). The classifier makes the pipeline up to six times cheaper (Tax), since rejected sentences never reach the mapper; on HMDA Checks, where most sentences are constraints, both variants cost the same. Low-cost backbones such as GPT-6 Luna and DeepSeek-V4-Flash cost a few cents per dataset.

7 RELATED WORK

Data Profiling. The closest line of work to `Norma` is *data profiling*, which discovers constraints, such as functional and conditional dependencies and DCs, directly from data instances [2, 9, 14, 20, 26, 42, 43, 47]. DCs subsume keys, FDs, and ordering constraints [14], which makes them our target formalism (Section 2). Approximate discovery tolerates violations in dirty data [38, 44], user-specific criteria help experts rank discovered rules [19], and LLMs now propose candidate rules from schemas and samples [8, 22, 31]. Still, these methods reveal what the observed data seems to satisfy, not whether a pattern is required by policy or regulation, so profiling cannot recover constraints that are already violated in the released data, and discovered DCs often include many spurious constraints [37]. Our goal is to recover constraints from normative text and then use them to validate the data.

Data Cleaning. Data cleaning systems use constraints, statistical signals, or both to detect and repair errors [1, 15, 36, 40, 48, 51], including rules with ML predicates (REEs) [6] and LLM-based detectors [41, 61]. Constraint-based methods are interpretable and auditable, but they assume that the relevant rules are already available. Data-driven methods reduce the need for manual rule specification, but without an external source of truth, they can struggle to distinguish rare valid values from genuine errors [40]. Our work obtains such rules from regulatory documents, as DCs over a controlled predicate space, which are executable, auditable, and checkable at scale [28, 35, 45].

Regulatory NLP. A separate line of work uses NLP to analyze regulations and compliance-related documents, for example, by extracting rights and obligations or by checking whether requirement statements comply with legal or policy text [10, 13, 17, 24, 29, 39]. LLMs now extract structured rules from regulations [23, 25, 27] and translate financial regulations into compliance code [32], although they still struggle to apply long sets of interdependent rules [57, 59]. These approaches remain text-centered or target a fixed application model, whereas we connect regulatory statements to a relational schema and translate them into constraints whose violations can be checked in the data.

Text-to-SQL. Text-to-SQL is also related, since it maps natural-language input to a SQL query over a database schema [4, 21, 30, 34, 49, 50, 52, 53, 55, 60]. However, Text-to-SQL answers a user question on the current instance, whereas `Norma` produces declarative invariants that must hold for all valid instances; beyond linking text to schema elements, it must also identify scope conditions and violation semantics. Closer in output, LLMs generate OCL constraints from specifications of UML models [3, 5], and Doc2DB systems verify extracted relations against integrity constraints [62]; in both cases, the target schema is designed together with the constraints, whereas `Norma` grounds regulatory text in an independently published dataset.

8 CONCLUSION

A constraint inferred from data describes what the data happens to satisfy, whereas a constraint contained in governing text describes what the data is required to satisfy. This distinction has received limited attention in database research, even though many published datasets are accompanied by documents that define data quality criteria at some level.

We showed that the gap between data and normative text can be addressed automatically: lightweight classification detects most checkable requirements, and a structured multi-step pipeline grounded in both text and schema translates them into executable constraints while avoiding hallucinated predicates and literals.

This enables a new form of validation: `Norma` can reveal compliance failures even when they do not manifest as discoverable patterns in the data. In such settings, the normative document becomes part of the data specification, and keeping that specification aligned with evolving text emerges as a concrete challenge for data management. Because each constraint comes from the text, it remains valid while the regulation is unchanged and can be re-executed, also incrementally [28], as the data evolves; only a change in the text requires a new mapping. Legal documents often have hundreds of sentences with nested clauses that must first be segmented and resolved, against at most 158 sentences in our datasets; since cost grows linearly with the number of sentences, we expect the pipeline to scale, but this remains to be shown.

REFERENCES

[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting data errors: where are we and what needs to be done? *PVLDB* 9, 12 (Aug. 2016), 993–1004. https://doi.org/10.14778/2994509.2994518

[2] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling relational data: a survey. *The VLDB Journal* 24, 4 (Aug. 2015), 557–581. https://doi.org/10.1007/s00778-015-0389-y

[3] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2023. On Codex Prompt Engineering for OCL Generation: An Empirical Study. In *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 148–157. https://doi.org/10.1109/MSR59073.2023.00033

[4] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. 2026. Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation. *Proc. VLDB Endow.* 19, 11 (2026), 3732–3744. https://doi.org/10.14778/3836663.3836721

[5] Hamza Attarwala, Moataz Chouchen, Mohammad Hamdaqa, and Omar Alam. 2026. Why Do LLMs Fail at OCL Generation? A Graph Reasoning Perspective. In *Proceedings of the 20th International Symposium on Empirical Software Engineering and Measurement (ESEM) (LIPIcs)*, Vol. 394. https://doi.org/10.4230/LIPIcs.ESEM.2026.48

[6] Xianchun Bao, Zian Bao, Binbin Bie, QingSong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, Mingliang Ouyang, Shuai Tang, Yaoshu Wang, Qiyuan Wei, Min Xie, Jing Zhang, Xin Zhang, Runxiao Zhao, and Shuping Zhou. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*. 106–119. https://doi.org/10.1145/3626246.3653372

[7] Carlo Batini, Cinzia Cappiello, Chiara Francalanci, and Andrea Maurino. 2009. Methodologies for data quality assessment and improvement. *ACM Comput. Surv.* 41, 3, Article 16 (July 2009), 52 pages. https://doi.org/10.1145/1541880.1541883

[8] Fabian Biester, Mohamed Abdelaal, and Daniel Del Gaudio. 2024. LLM-Clean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs. *CoRR* abs/2404.18681 (2024).

[9] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient Denial Constraint Discovery with Hydra. *PVLDB* 11, 3 (2017), 311–323.

[10] Travis D. Breaux, Matthew W. Vail, and Annie I. Antón. 2006. Towards Regulatory Compliance: Extracting Rights and Obligations to Align Requirements with Regulations. In *Proceedings of the 14th IEEE International Requirements Engineering Conference (RE)*. IEEE Computer Society, 46–55.

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[12] Ismael Caballero, Fernando Gualo, Moisés Rodríguez, and Mario Piattini. 2022. BR4DQ: A methodology for grouping business rules for data quality evaluation. *Information Systems* 109 (2022), 102058. https://doi.org/10.1016/j.is.2022.102058

[13] Orlando Amaral Cejas, Muhammad Ilyas Azeem, Sallam Abualhaija, and Lionel C. Briand. 2023. NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR. *IEEE Trans. Softw. Eng.* 49, 9 (Sept. 2023), 4282–4303. https://doi.org/10.1109/TSE.2023.3288901

[14] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *PVLDB* 6, 13 (Aug. 2013), 1498–1509. https://doi.org/10.14778/2536258.2536262

[15] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 458–469. https://doi.org/10.1109/ICDE.2013.6544847

[16] Kevin L. Cope. 2023. Measuring law's normative force. *Journal of Empirical Legal Studies* 20, 4 (2023), 1005–1044. https://doi.org/10.1111/jels.12364 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/jels.12364

[17] Souvick Das, Novarun Deb, Nabendu Chaki, and Agostino Cortesi. 2025. A Multi-Agent RAG Framework for Regulatory Compliance Checking of Software Requirements. *ACM Trans. Softw. Eng. Methodol.* (Dec. 2025). https://doi.org/10.1145/3785472 Just Accepted.

[18] Ruanitto Docini, Eduardo C. de Almeida, and Luiz S. Oliveira. 2025. The Wrecking SQL Incremental Validation Methodology. In *Database and Expert Systems Applications - 36th International Conference, DEXA 2025, Bangkok, Thailand, August 25-27, 2025, Proceedings, Part II (Lecture Notes in Computer Science)*. Springer, 376–382. https://doi.org/10.1007/978-3-032-02088-8_30

[19] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proc. ACM Manag. Data* 1, 1 (2023), 1–29. https://doi.org/10.1145/3588924

[20] Wenfei Fan, Jianhua Li, Shuai Ma, and Nan Tang. 2009. Discovering Conditional Functional Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1231–1234.

[21] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[22] Anna-Christina Glock, Thomas Hütter, Johannes Fürnkranz, Wolfram Wöß, Christine Dominka-Kiss, and Lisa Ehrlinger. 2026. Data Quality Rule Generation with LLMs. *CoRR* abs/2609.06053 (2026).

[23] Keerat Guliani, Deepkamal Gill, David Landsman, Nima Eshraghi, Krishna Kumar, and Lovedeep Gondara. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. *CoRR* abs/2604.02276 (2026).

[24] Shabnam Hassani, Mehrdad Sabetzadeh, Daniel Amyot, and Jain Liao. 2024. Rethinking Legal Compliance Automation: Opportunities with Large Language Models. In *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE)*. 432–440. https://doi.org/10.1109/RE59067.2024.00051

[25] Hansi Hettiarachchi, Amna Dridi, Mohamed Medhat Gaber, Pouyan Parsafard, Nicoleta Bocaneala, Katja Breitenfelder, Gonçal Costa, Maria Hedblom, Mihaela Juganaru-Mathieu, Thamer Mecharnia, Sumee Park, He Tan, Abdel-Rahman H. Tawil, and Edlira Vakaj. 2025. CODE-ACCORD: A Corpus of building regulatory data for rule generation towards automatic compliance checking. *Scientific Data* 12, 1 (2025), 170.

[26] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1998. TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 392–401.

[27] Zarina Kabzhan, Alexandr Shakhnovich, Sergey Gorshkov, Yussuf Yemenov, Fedor Gorshkov, and Nazym Shogelova. 2025. Semantic and ontology-based analysis of regulatory documents for construction industry digitalization. *Frontiers in Built Environment* 11 (2025), 1575913. https://doi.org/10.3389/fbuil.2025.1575913

[28] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2025. Incremental Detection of Denial Constraint Violations. *PVLDB* 18, 4 (May 2025), 1000–1012. https://doi.org/10.14778/3717755.3717761

[29] Nadzeya Kiyavitskaya, Nicola Zeni, Travis D. Breaux, Annie I. Antón, James R. Cordy, Luisa Mich, and John Mylopoulos. 2008. Automating the Extraction of Rights and Obligations for Regulatory Compliance. In *Conceptual Modeling - ER 2008, 27th International Conference on Conceptual Modeling, Barcelona, Spain, October 20-24, 2008. Proceedings (Lecture Notes in Computer Science)*, Vol. 5231. Springer, 154–168.

[30] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1466, 9 pages. https://doi.org/10.1609/aaai.v37i11.26535

[31] Qing Li, Yanyan Shen, Qibin Zheng, Yi Liu, and Xingchun Diao. 2026. LCATS: LLM-Guided Constraint-Aware Tabular Data Synthesis. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD)*. 2698–2707. https://doi.org/10.1145/3770855.3818181

[32] Siyuan Li, Jian Chen, Rui Yao, Xuming Hu, Peilin Zhou, Weihua Qiu, Simin Zhang, Chucheng Dong, Zhiyao Li, Qipeng Xie, and Zixuan Yuan. 2025. Compliance-to-Code: Enhancing Financial Compliance Checking via Code Generation. *CoRR* abs/2505.19804 (2025).

[33] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 4870–4888. https://doi.org/10.18653/v1/2020.findings-emnlp.438

[34] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, and Jingren Zhou. 2026. XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering* (2026), 1–14. https://doi.org/10.1109/TKDE.2026.3657851

[35] Zifan Liu, Shaleen Deep, Anna Fariha, Fotis Psallidas, Ashish Tiwari, and Avrilia Floratou. 2024. Rapidash: Efficient Detection of Constraint Violations. *Proc. VLDB Endow.* 17, 8 (April 2024), 2009–2021. https://doi.org/10.14778/3659437.3659454

[36] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 865–882. https://doi.org/10.1145/3299869.3324956

[37] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2025. How and Why False Denial Constraints are Discovered. *PVLDB* 18, 10 (2025), 3477–3489.

[38] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2026. Discovering Approximate Denial Constraints in Large Databases. *Proc. VLDB Endow.* 19, 9 (2026), 2086–2098. https://doi.org/10.14778/3819518.3819536

[39] Arsalan Masoudifard, Mohammad Mowlavi Sorond, Moein Madadi, Mohammad Sabokrou, and Elahe Habibi. 2024. Leveraging Graph-RAG and Prompt Engineering to Enhance LLM-Based Automated Requirement Traceability and Compliance Checks. *CoRR* abs/2412.08593 (2024).

[40] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *PVLDB* 17, 10 (June 2024), 2617–2630. https://doi.org/10.14778/3675034.3675051

[41] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Yaoshu Wang, and Jianwei Yin. 2025. ZeroED: Hybrid Zero-Shot Error Detection Through Large Language Model Reasoning. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*. 3126–3139. https://doi.org/10.1109/ICDE65448.2025.00234

[42] Thorsten Papenbrock and Felix Naumann. 2016. A Hybrid Approach to Functional Dependency Discovery. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 821–833.

[43] Eduardo Pena, Fabio Porto, and Felix Naumann. 2024. Discovering Denial Constraints in Dynamic Datasets. 3546–3558. https://doi.org/10.1109/ICDE60146.2024.00273

[44] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2019. Discovery of Approximate (and Exact) Denial Constraints. *PVLDB* 13, 3 (2019), 266–278.

[45] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2021. Fast detection of denial constraint violations. *PVLDB* 15, 4 (Dec. 2021), 859–871. https://doi.org/10.14778/3503585.3503595

[46] Eduardo H. M. Pena, Junio de Freitas, Eduardo Cunha de Almeida, and Altigran da Silva. 2026. Text-to-Constraints: Mapping Regulatory Requirements to Executable Data Quality Constraints (Extended Version). https://github.com/EduardoPena/norma-text-to-constraints.

[47] Eduardo H. M. Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *PVLDB* 16, 4 (2022), 684–696.

[48] Jinfeng Peng, Derong Shen, Nan Tang, Tieying Liu, Yue Kou, Tiezheng Nie, Hang Cui, and Ge Yu. 2022. Self-Supervised and Interpretable Data Cleaning with Sequence Generative Adversarial Networks. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 433–446. https://doi.org/10.14778/3570690.3570694

[49] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1577, 10 pages.

[50] Ricardo Trainotti Rabonato and Lilian Berton. 2026. The Impact of Language and Portuguese Intralinguistic Variation on NL2SQL. In *Anais do XVII Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana (STIL)*. 317–330. https://doi.org/10.5753/stil.2026.26611

[51] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: holistic data repairs with probabilistic inference. *PVLDB* 10, 11 (Aug. 2017), 1190–1201. https://doi.org/10.14778/3137628.3137631

[52] Tonghui Ren, Chen Ke, Yuankai Fan, Yinan Jing, Zhenying He, Kai Zhang, and X. Sean Wang. 2025. The Power of Constraints in Natural Language to SQL Translation. *Proc. VLDB Endow.* 18, 7 (March 2025), 2097–2111. https://doi.org/10.14778/3734839.3734847

[53] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. https://doi.org/10.18653/v1/2021.emnlp-main.779

[54] Flavia Serra, Verónika Peralta, Adriana Marotta, and Patrick Marcel. 2024. Use of Context in Data Quality Management: A Systematic Literature Review. *J. Data and Information Quality* 16, 3, Article 19 (Oct. 2024), 41 pages. https://doi.org/10.1145/3672082

[55] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.* 58, 2, Article 54 (Sept. 2025), 37 pages. https://doi.org/10.1145/3737873

[56] Shaoxu Song, Fei Gao, Ruihong Huang, and Chaokun Wang. 2020. Data Dependencies over Big Data: A Family Tree. *IEEE Transactions on Knowledge and Data Engineering* PP (12 2020), 1–1. https://doi.org/10.1109/TKDE.2020.3046443

[57] Utkarsh Soni, Syed Shariyar Murtaza, Yifan Nie, Sachin Chandrasekhar, and Eugene Wen. 2026. Tasks over Application Manuals: Revealing Gaps in Long-Horizon Procedural Reasoning for Language Models. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM)*. https://doi.org/10.1145/3799682.3841039

[58] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. 2023. Text Classification via Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 8990–9005. https://doi.org/10.18653/v1/2023.findings-emnlp.603

[59] Bohan Yu, Pengfei Cao, Chen Han, Chenxi Zhou, Zhiheng Zhang, Zhiyang Xie, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. 2026. Beyond Factual Knowledge: Benchmarking and Learning Step-Level Procedural Rule Reasoning in Large Language Models. *CoRR* abs/2608.22753 (2026).

[60] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Brussels, Belgium, 3911–3921.

[61] Shuo Zhang, Zezhou Huang, and Eugene Wu. 2024. Data Cleaning Using Large Language Models. *CoRR* abs/2410.15547 (2024).

[62] Zhengxuan Zhang, Zhuowen Liang, Jiazhuo Chen, Haixun Wang, and Nan Tang. 2026. Document-to-Database: Extraction Meets Relational Semantics. *Proc. VLDB Endow.* 19, 9 (2026), 2522–2532. https://doi.org/10.14778/3819518.3819568

[63] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29